

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 8: DYNAMIC PROGRAMMING

Abdullah Bal, PhD

Objectives



Recap

Space-Time Trade-offs

Counting sort

Horspool String Matching

Hashing

Open Hashing

Closed Hashing



Dynamic programming



Change making Coin collecting



Floy'sd Algorithm

Problem

- How many coins, at a minimum, are required to achieve a total of 6?”
- What coins meet the condition mentioned above?



Dynamic Programming

- Invented by American mathematician **Richard Bellman** in the **1950s** to solve multistage decision processes and later assimilated by CS
- Dynamic Programming is a general algorithm design technique for solving problems defined by **recurrences with overlapping subproblems**
- Rather than solving overlapping subproblems **again and again**, dynamic programming suggests solving each of the smaller subproblems **only once and recording the results** in a table from which a solution to the original problem can then be obtained.
- Although a straightforward application of dynamic programming can be interpreted as a special variety of space-for-time trade-off, a dynamic programming algorithm can sometimes be refined **to avoid using extra space**.

Dynamic Programming

- Main idea:
 - set up a recurrence relating a solution to a larger instance to solutions of some smaller instances
 - solve smaller instances once
 - record solutions in a table
 - extract solution to the initial instance from that table

Examples of DP algorithms

- General case of the **change making problem**
- Some difficult discrete optimization problems:
 - **Knapsack**
 - Traveling salesman
- Warshall's algorithm for **transitive closure**
- Floyd's algorithm for **all-pairs shortest paths**
- Constructing an optimal binary search tree

Change Making Problem



Change Making Problem

- Give change for amount n using the minimum number of coins of denominations
- We consider a dynamic programming algorithm for the general case, assuming availability of unlimited quantities of coins for each of the m denominations

$$d_1 < d_2 < \dots < d_m$$

Change Making Problem

- Let $F(n)$ be the minimum number of coins whose values add up to n ; it is convenient to define

$$F(0) = 0$$

- The amount n can only be obtained by adding one coin of denomination

$$d_j \text{ to the amount } n - d_j \quad \text{for } j = 1, 2, \dots, m \quad n \geq d_j$$

- Therefore, we can consider all such denominations and select the one minimizing

$$F(n - d_j) + 1$$

- Since 1 is a constant, we can, find the smallest $F(n - d_j)$ first and then add 1 to it.

Change Making Problem

- We have the following recurrence for $F(n)$:

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1 \quad \text{for } n > 0,$$

$$F(0) = 0.$$

- We can compute $F(n)$ by filling a **one-row table** left to right,
- Computing a table entry here requires finding the minimum of up to m numbers.

Example 1: Change Making Problem

- The application of the algorithm to amount $n = 6$ and denominations 1, 3, 4

$$F[0] = 0$$

n	0	1	2	3	4	5	6
F	0						

$$F[1] = \min\{F[1 - 1]\} + 1 = 1$$

n	0	1	2	3	4	5	6
F	0	1					

$$F[2] = \min\{F[2 - 1]\} + 1 = 2$$

n	0	1	2	3	4	5	6
F	0	1	2				

$$F[3] = \min\{F[3 - 1], F[3 - 3]\} + 1 = 1$$

n	0	1	2	3	4	5	6
F	0	1	2	1			

$$F[4] = \min\{F[4 - 1], F[4 - 3], F[4 - 4]\} + 1 = 1$$

n	0	1	2	3	4	5	6
F	0	1	2	1	1		

$$F[5] = \min\{F[5 - 1], F[5 - 3], F[5 - 4]\} + 1 = 2$$

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	

$$F[6] = \min\{F[6 - 1], F[6 - 3], F[6 - 4]\} + 1 = 2$$

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	2



$n=6$

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1 \quad \text{for } n > 0,$$

$$F(0) = 0.$$

Result: $3 + 3 = 6$

Example 1: Change Making Problem

- The answer it yields is two coins.
- To find the coins of an optimal solution, we need to **backtrace** the computations to see which of the denominations produced the minima in the formula.
- For the instance considered, the last application of the formula (for $n = 6$), the minimum was produced by $d_2 = 3$.
- The second minimum (for $n = 6 - 3$) was also produced for a coin of that denomination.
- Thus, the minimum-coin set for $n = 6$ is two 3's.

Example 2: Change Making Problem

- The application of the algorithm to amount $n = 7$ and denominations 1, 2, 4

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1 \quad \text{for } n > 0,$$

$$F(0) = 0.$$

n	0	1	2	3	4	5	6	7
F	0							



$n=7$

Example 2: Change Making Problem

- The application of the algorithm to amount $n = 7$ and denominations 1, 2, 4

$$F(0)=0$$

$$F(1)=\min\{F(1-1)\}+1=1$$

$$F(2)=\min\{F(2-1), F(2-2)\}+1=1$$

$$F(3)=\min\{F(3-1), F(3-2)\}+1=2$$

$$F(4)=\min\{F(4-1), F(4-2), F(4-4)\}+1=1$$

$$F(5)=\min\{F(5-1), F(5-2), F(5-4)\}+1=2$$

$$F(6)=\min\{F(6-1), F(6-2), F(6-4)\}+1=2$$

$$F(7)=\min\{F(7-1), F(7-2), F(7-4)\}+1=3$$

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1 \quad \text{for } n > 0,$$

$$F(0) = 0.$$

n	0	1	2	3	4	5	6	7
F	0	1	1	2	1	2	2	3

Result: $4 + 2 + 1 = 7$



$n=7$

Example 3: Change Making Problem

- The application of the algorithm to amount $n = 10$ and denominations 1, 5, 6, 9

$$F(0)=0$$

$$F(1)=\min\{F(1-1)\}+1=1$$

$$F(2)=\min\{F(2-1)\}+1=2$$

$$F(3)=\min\{F(3-1)\}+1=3$$

$$F(4)=\min\{F(4-1)\}+1=4$$

$$F(5)=\min\{F(5-1), F(5-5)\}+1=1$$

$$F(6)=\min\{F(6-1), F(6-5), F(6-6)\}+1=2$$

$$F(7)=\min\{F(7-1), F(7-5), F(7-6)\}+1=2$$

$$F(8)=\min\{F(8-1), F(8-5), F(8-6)\}+1=3$$

$$F(9)=\min\{F(9-1), F(9-5), F(9-6), F(9-9)\}+1=1$$

$$F(10)=\min\{F(10-1), F(10-5), F(10-6), F(10-9)\}+1=2$$

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1 \quad \text{for } n > 0,$$

$$F(0) = 0.$$

n	0	1	2	3	4	5	6	7	8	9	10
F	0	1	2	3	4	1	2	2	3	1	2

Result: **9 + 1**

or

5 + 5



$n=10$

Change Making Problem

ALGORITHM *ChangeMaking*($D[1..m], n$)

//Applies dynamic programming to find the minimum number of coins
//of denominations $d_1 < d_2 < \dots < d_m$ where $d_1 = 1$ that add up to a
//given amount n

//Input: Positive integer n and array $D[1..m]$ of increasing positive
// integers indicating the coin denominations where $D[1] = 1$

//Output: The minimum number of coins that add up to n

$F[0] \leftarrow 0$

for $i \leftarrow 1$ **to** n **do**

$temp \leftarrow \infty; j \leftarrow 1$

while $j \leq m$ **and** $i \geq D[j]$ **do**

$temp \leftarrow \min(F[i - D[j]], temp)$

$j \leftarrow j + 1$

$F[i] \leftarrow temp + 1$

return $F[n]$

- The time efficiency of the algorithm is $\Theta(nm)$.



Coin-collecting by robot

- Several coins are placed in cells of an $n \times m$ board, **no more than one coin per cell**.
- A robot, located in the upper left cell of the board, needs to collect **as many of the coins as possible** and bring them to the bottom right cell.
- On each step, the robot can move either **one cell to the right or one cell down** from its current location.
- When the robot visits a cell with a coin, it **always picks up** that coin.
- Design an algorithm to find the **maximum number** of coins the robot can collect and **a path it needs to follow to do this**.

	1	2	3	4	5	6
1					●	
2		●		●		
3				●		●
4			●			●
5	●				●	

Coin-collecting problem

- Let $F(i,j)$ be the largest number of coins the robot can collect and bring to cell (i,j) in the i th row and j th column.
- The largest number of coins that can be brought to cell (i,j) :
 - from the left neighbor ?
 - from the neighbor above?

- The recurrence:

$$F(i, j) = \max\{F(i-1, j), F(i, j-1)\} + c_{ij} \quad \text{for } 1 \leq i \leq n, \quad 1 \leq j \leq m$$

$$F(0, j) = 0 \quad \text{for } 1 \leq j \leq m \quad \text{and} \quad F(i, 0) = 0 \quad \text{for } 1 \leq i \leq n,$$

where c_{ij} = value of the coin, if there is a coin in cell (i,j)

$c_{ij} = 0$ otherwise

Coin-collecting problem

- Tracing the computations backward makes it possible to get an optimal path:
- If $F(i - 1, j) > F(i, j - 1)$, an optimal path to cell (i, j) must come down from the adjacent cell above it;
- If $F(i - 1, j) < F(i, j - 1)$, an optimal path to cell (i, j) must come from the adjacent cell on the left;
- If $F(i - 1, j) = F(i, j - 1)$, it can reach cell (i, j) from either direction.
- This yields two optimal paths for the instance.

Example 1: Coin-collecting problem

- We can fill in the $n \times m$ table of $F(i, j)$ values either row by row or column by column, as is typical for dynamic programming algorithms involving two-dimensional tables.

	1	2	3	4	5	6
1					1	
2		1		1		
3				1		1
4			1			1
5	1				1	

Coins to collect

	1	2	3	4	5	6
1	0	0	0	0	1	1
2	0	1	1	2	2	2
3	0	1	1	3	3	4
4	0	1	2	3	3	5
5	1	1	2	3	4	5

Dynamic programming algorithm results

	1	2	3	4	5	6
1	1	1			1	
2	1	1	1	1		
3				1	1	1
4			1			1
5	1				1	

Two paths to collect 5 coins, the maximum number of coins possible.

$$F(i, j) = \max\{F(i-1, j), F(i, j-1)\} + c_{ij} \quad \text{for } 1 \leq i \leq n, \quad 1 \leq j \leq m$$

$$F(0, j) = 0 \quad \text{for } 1 \leq j \leq m \quad \text{and} \quad F(i, 0) = 0 \quad \text{for } 1 \leq i \leq n,$$

Example 2: Coin-collecting problem

	1	2	3	4	5	6
1					 6	
2		 5		 4		
3				 3		 8
4			 9			 3
5	 10				 9	

Example 2: Solution of the coin-collecting problem

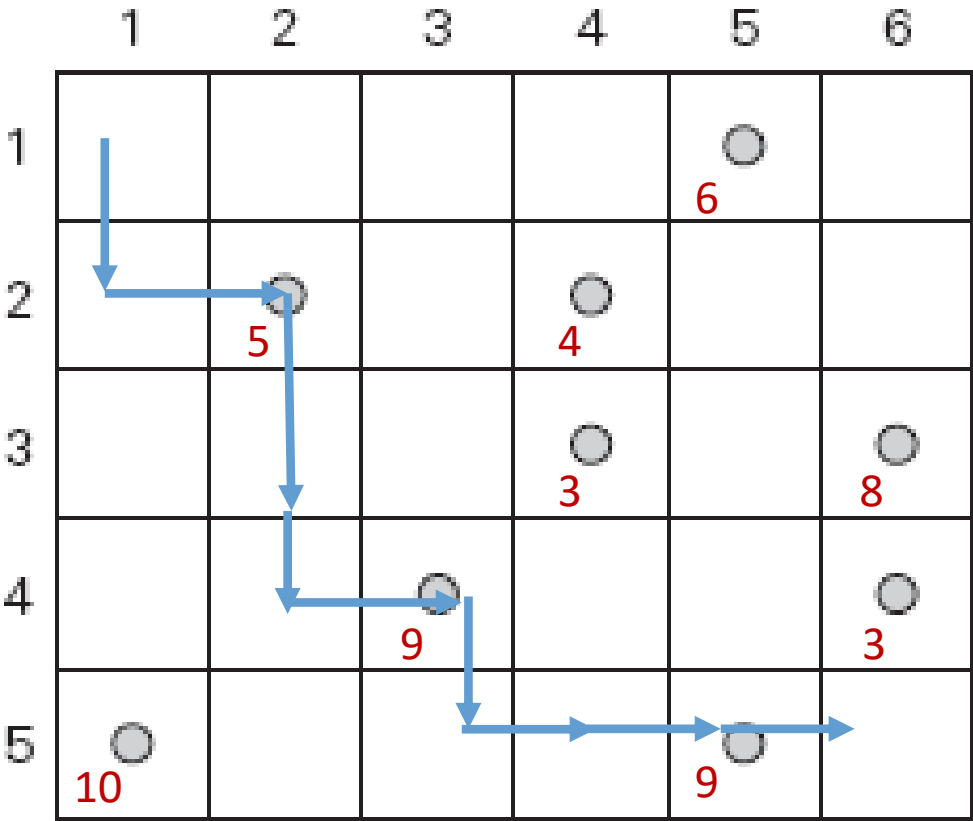
	1	2	3	4	5	6
1					 6	
2		 5		 4		
3				 3		 8
4			 9			 3
5	 10				 9	

	1	2	3	4	5	6
1	0	0	0	0	6	6
2	0	5	5	9	9	9
3	0	5	5	12	12	20
4	0	5	14	14	14	23
5	10	10	14	14	23	23

Example 2: Solution of the coin-collecting problem

	1	2	3	4	5	6
1	0	0	0	0	6	6
2	0	5	5	9	9	9
3	0	5	5	12	12	20
4	0	5	14	14	14	23
5	10	10	14	14	23	23

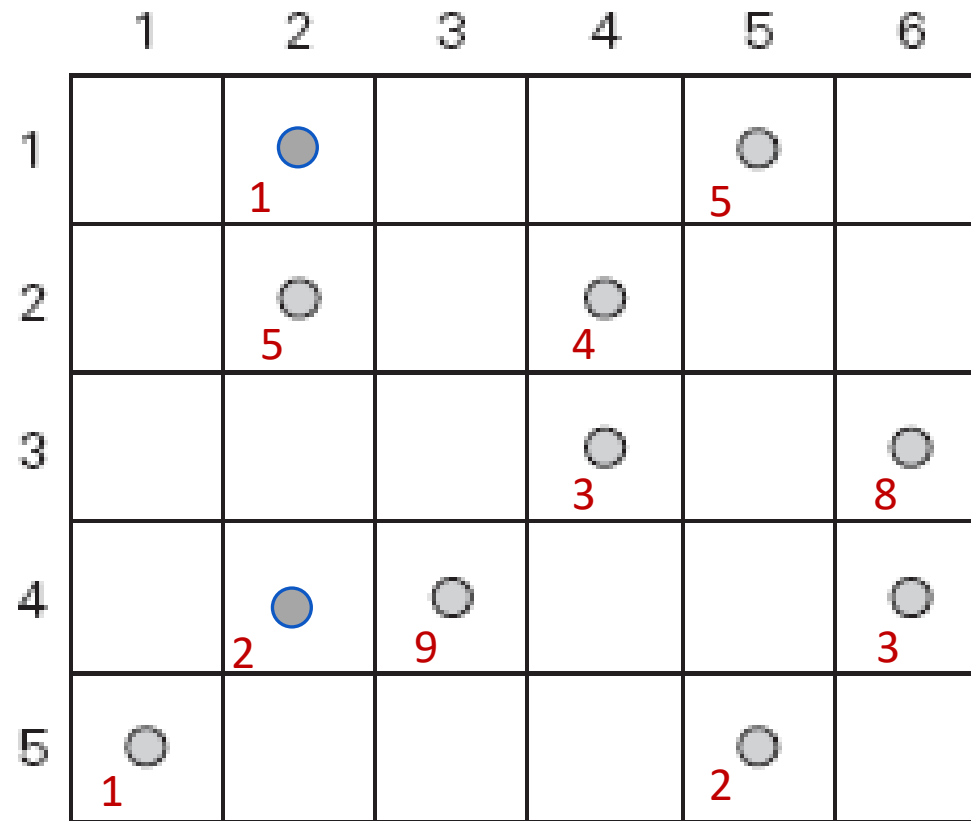
Two equal routs are possible.



Total: $5 + 9 + 9 = 23$ or

Total: $3 + 8 + 3 + 4 + 5 = 23$

Example 3: Coin-collecting problem



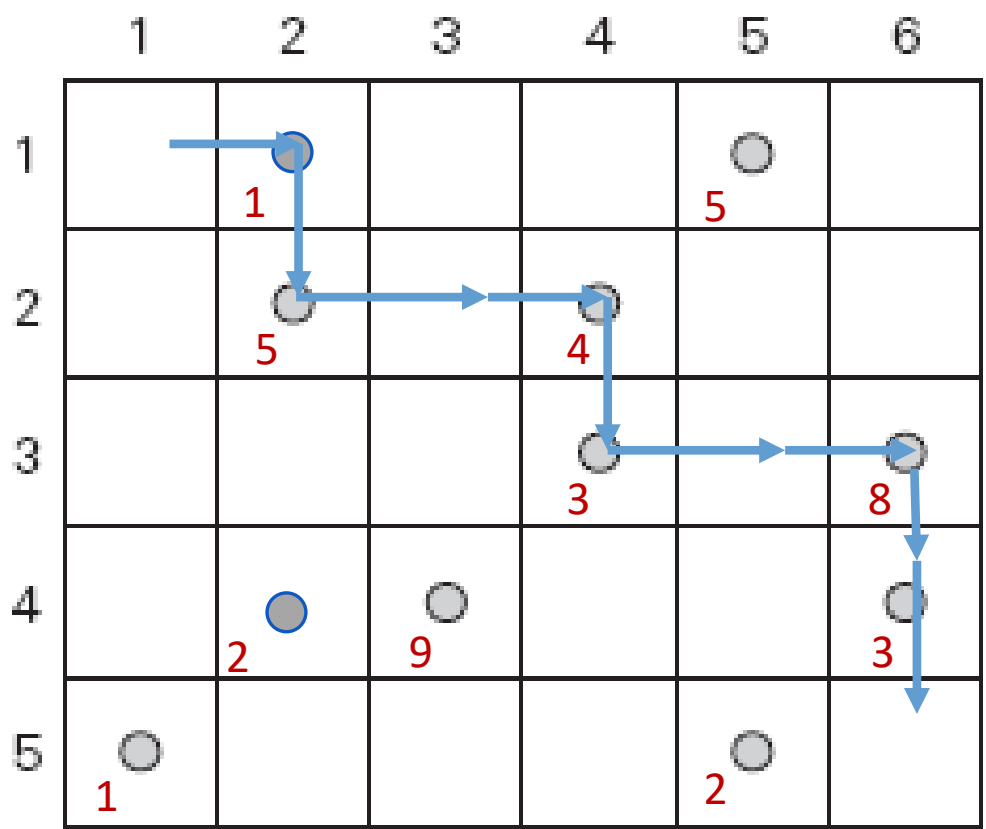
Example 3: Solution of the coin-collecting problem

	1	2	3	4	5	6
1		 1			 5	
2		 5		 4		
3				 3		 8
4		 2	 9			 3
5	 1				 2	

	1	2	3	4	5	6
1	0	1	1	1	6	6
2	0	6	6	10	10	10
3	0	6	6	13	13	21
4	0	8	17	17	17	24
5	1	8	17	17	19	24

Example 3: Solution of the coin-collecting problem

	1	2	3	4	5	6
1	0	1	1	1	6	6
2	0	6	6	10	10	10
3	0	6	6	13	13	21
4	0	8	17	17	17	24
5	1	8	17	17	19	24



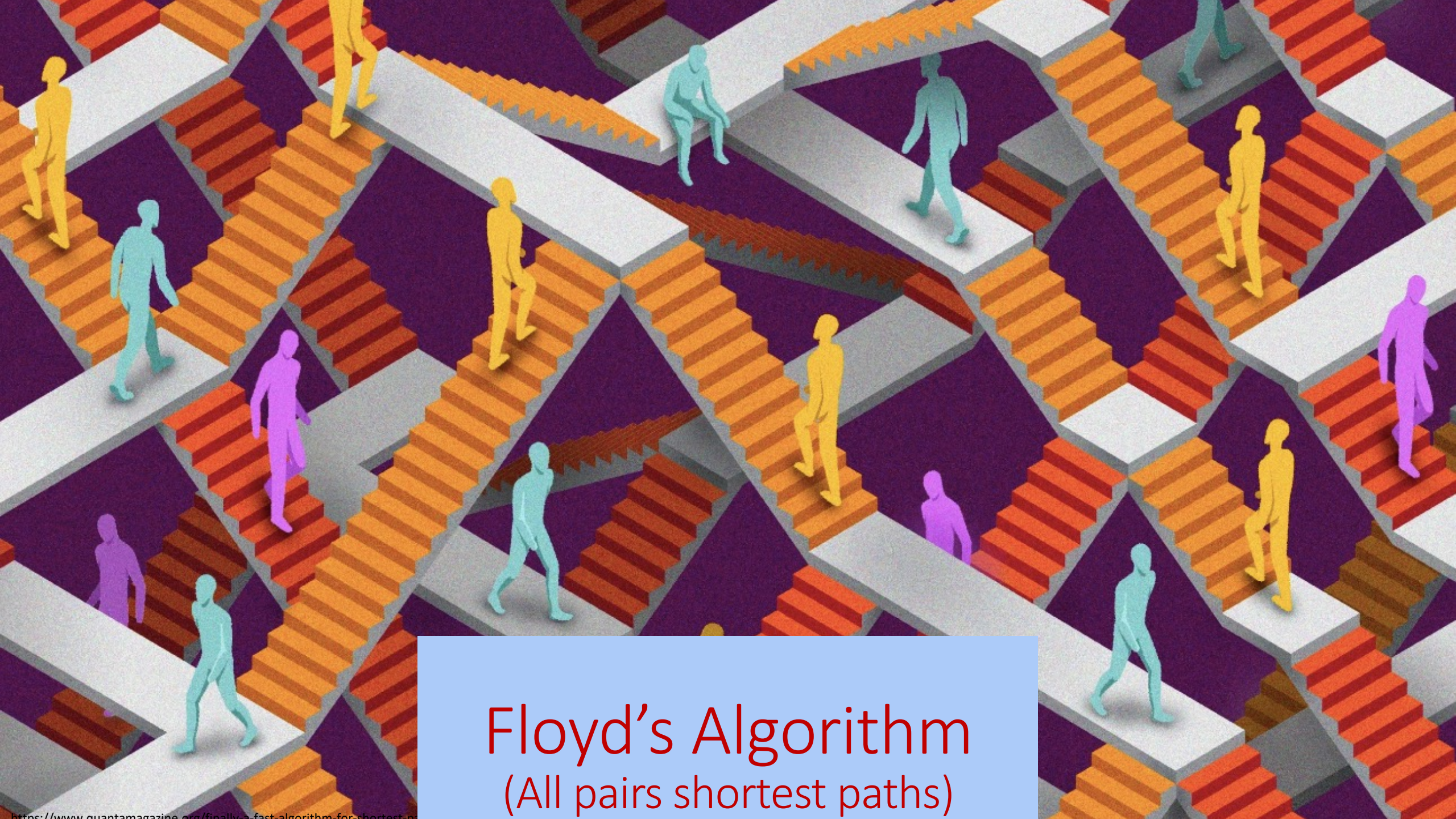
Total: $1 + 5 + 4 + 3 + 8 + 3 = 24$

The coin-collecting problem

- Since computing the value of $F(i, j)$ by the formula for each cell of the table takes constant time, the time efficiency of the algorithm is $\Theta(nm)$.
- Its space efficiency is, obviously, also $\Theta(nm)$.

ALGORITHM *RobotCoinCollection*($C[1..n, 1..m]$)

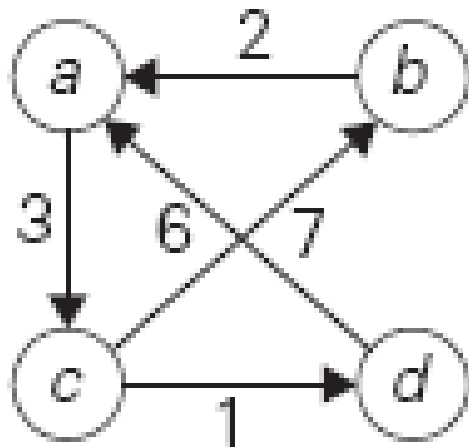
```
//Applies dynamic programming to compute the largest number of
//coins a robot can collect on an  $n \times m$  board by starting at (1, 1)
//and moving right and down from upper left to down right corner
//Input: Matrix  $C[1..n, 1..m]$  whose elements are equal to 1 and 0
//for cells with and without a coin, respectively
//Output: Largest number of coins the robot can bring to cell  $(n, m)$ 
 $F[1, 1] \leftarrow C[1, 1]$ ; for  $j \leftarrow 2$  to  $m$  do  $F[1, j] \leftarrow F[1, j - 1] + C[1, j]$ 
for  $i \leftarrow 2$  to  $n$  do
     $F[i, 1] \leftarrow F[i - 1, 1] + C[i, 1]$ 
    for  $j \leftarrow 2$  to  $m$  do
         $F[i, j] \leftarrow \max(F[i - 1, j], F[i, j - 1]) + C[i, j]$ 
return  $F[n, m]$ 
```



Floyd's Algorithm

(All pairs shortest paths)

All Pairs Shortest Paths Problem



$$W = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

$$D = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 7 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix} \end{matrix}$$

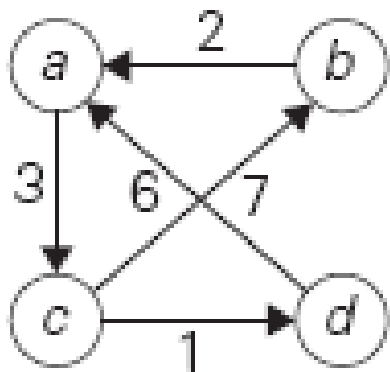
C → a ?

All pairs shortest paths

- Given a weighted connected graph (**undirected or directed**), the ***all-pairs shortest-paths problem*** asks to find the distances—i.e., the **lengths of the shortest paths**— **from each vertex to all other vertices**.
- Because of its important applications to communications, transportation networks, and operations research, it has been thoroughly studied over the years.
- Among recent applications of the all-pairs shortest-path problem is **precomputing distances** for motion planning **in computer games**.

All pairs shortest paths

- It is convenient to record the lengths of shortest paths in an $n \times n$ matrix D called the ***distance matrix***:
- The element d_{ij} in the i^{th} row and the j^{th} column of this matrix indicates the length of the shortest path from the i^{th} vertex to the j^{th} vertex.



$$W = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

$$D = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 7 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix} \end{matrix}$$

Floyd's Algorithm: All pairs shortest paths

- We can generate the distance matrix with an algorithm that is very similar to Warshall's algorithm.
- It is called ***Floyd's algorithm*** after its co-inventor Robert W. Floyd.
- The algorithm can be enhanced to find not only the lengths of the shortest paths for all vertex pairs but also **the shortest paths themselves**.

Floyd's Algorithm: All pairs shortest paths

- Floyd's algorithm computes the distance matrix of a weighted graph with n vertices through a series of $n \times n$ matrices:

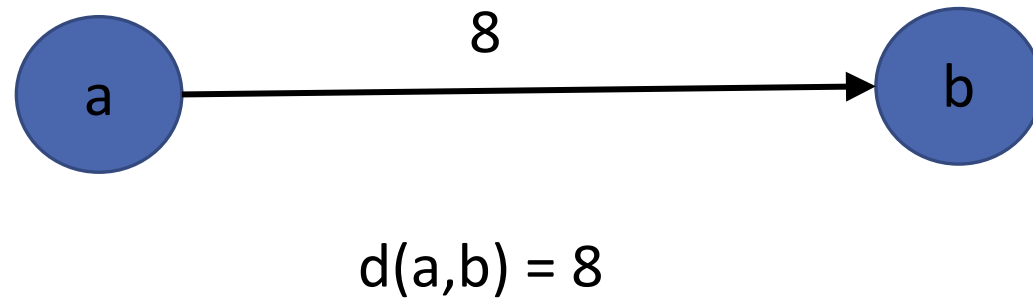
$$D^{(0)}, \dots, D^{(k-1)}, D^{(k)}, \dots, D^{(n)}.$$

- On the k -th iteration, the algorithm determines shortest paths between every pair of vertices i, j that use only vertices among $1, \dots, k$ as intermediate

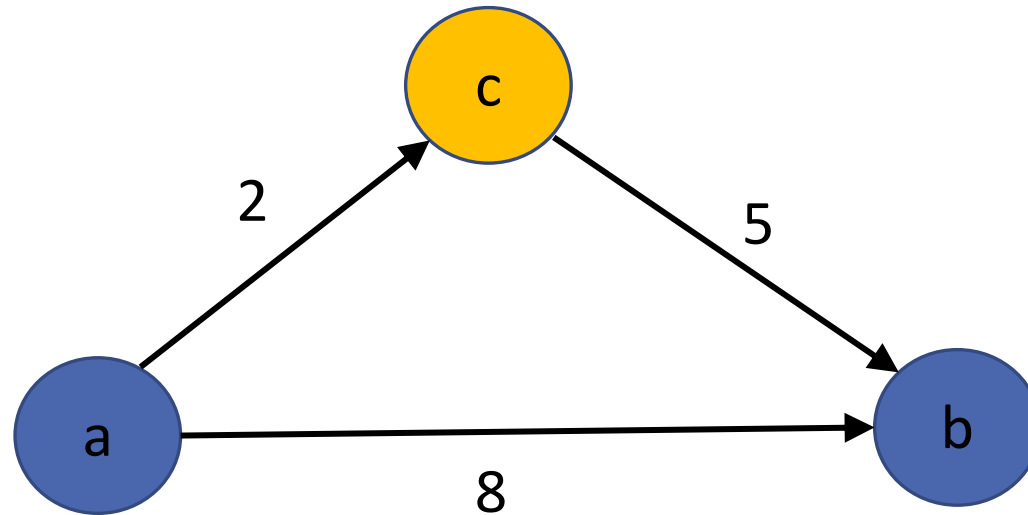
$$D^{(k)}[i,j] = \min \{D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j]\}$$

- We can compute all the elements of each matrix $D^{(k)}$ from its immediate predecessor $D^{(k-1)}$

Floyd's Algorithm: All pairs shortest paths



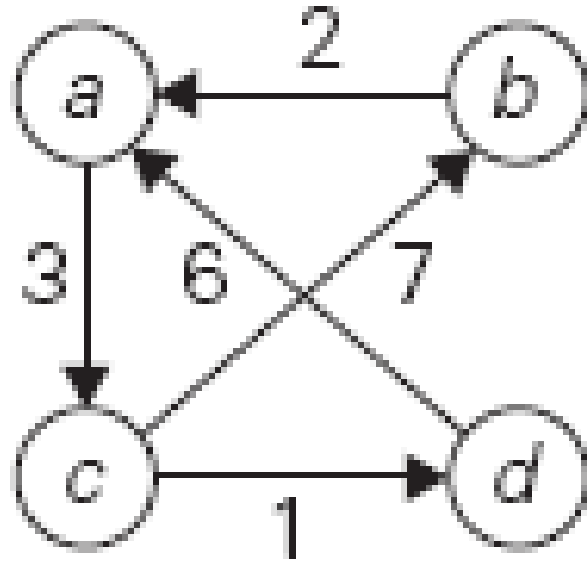
Floyd's Algorithm: All pairs shortest paths



$$\begin{aligned} d(a,b) &= \min\{d(a,b) , (d(a,c) + d(c,b)) \} \\ &= \min\{8 , (2+5) \}=7 \end{aligned}$$

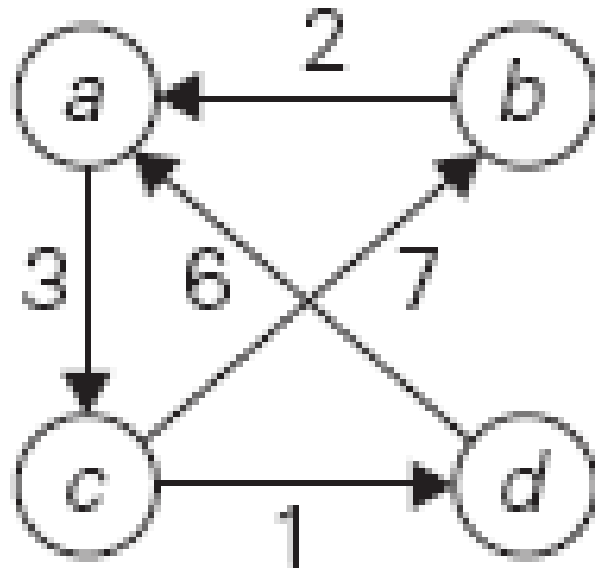
Example 1: Floyd's Algorithm

- In a weighted (di)graph, find shortest paths between every pair of vertices



Example 1: Floyd's Algorithm

- Step 1: Generate weight matrix $D^{(0)}$ (Adjacency matrix).



$D^{(0)} =$

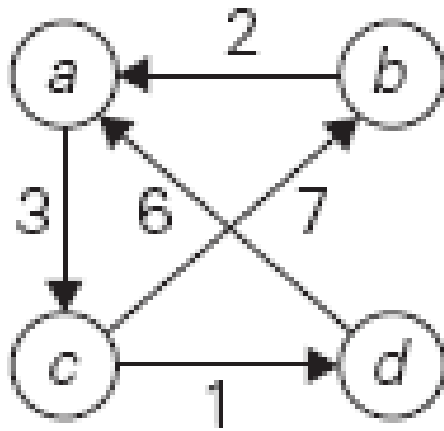
	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

Formula of D:

$$D^{(k)}[i,j] = \min \{D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j]\}$$

Example 1: Floyd's Algorithm

Step 2: Generate $D^{(1)}$ considering the shortest path for each vertex through vertex **a**.



$D^{(0)} =$

	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

$D^{(1)} =$

	a	b	c	d
a	0	∞	3	∞
b	2	0	5	∞
c	∞	7	0	1
d	6	∞	9	0

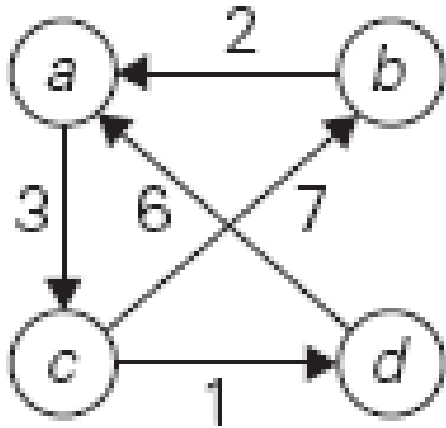
- $b, c = \min\{(b \rightarrow c), (b \rightarrow a + a \rightarrow c)\} = \min\{\infty, 2 + 3\} = 5$
- $b, d = \min\{(b \rightarrow d), (b \rightarrow a + a \rightarrow d)\} = \min\{\infty, 2 + \infty\} = \infty$

- $c, b = \min\{(c \rightarrow b), (c \rightarrow a + a \rightarrow b)\} = \min\{7, \infty + \infty\} = 7$
- $c, d = \min\{(c \rightarrow d), (c \rightarrow a + a \rightarrow d)\} = \min\{1, \infty + \infty\} = 1$

- $d, b = \min\{(d \rightarrow b), (d \rightarrow a + a \rightarrow b)\}$
 $= \min\{\infty, 6 + \infty\} = \infty$
- $d, c = \min\{(d \rightarrow c), (d \rightarrow a + a \rightarrow c)\}$
 $= \min\{\infty, 6 + 3\} = 9$

Example 1: Floyd's Algorithm

Step 3: Generate $D^{(2)}$ considering the shortest path for each vertex through vertex **b**.



$D^{(1)} =$

	a	b	c	d
a	0	∞	3	∞
b	2	0	5	∞
c	∞	7	0	1
d	6	∞	9	0

$D^{(2)} =$

	a	b	c	d
a	0	∞	3	∞
b	2	0	5	∞
c	9	7	0	1
d	6	∞	9	0

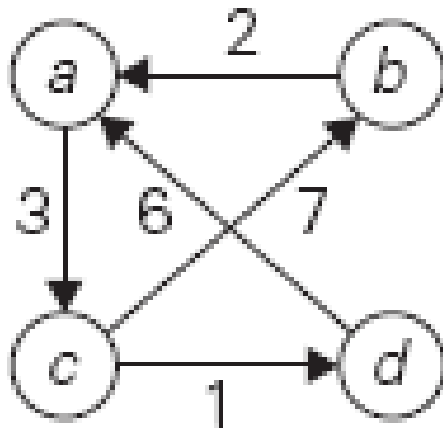
- $a, c = \min\{(a \rightarrow c), (a \rightarrow b + b \rightarrow c)\} = \min\{3, \infty + 5\} = 3$
- $a, d = \min\{(a \rightarrow d), (a \rightarrow b + b \rightarrow d)\} = \min\{\infty, \infty + \infty\} = \infty$

- $c, a = \min\{(c \rightarrow a), (c \rightarrow b + b \rightarrow a)\} = \min\{\infty, 7 + 2\} = 9$
- $c, d = \min\{(c \rightarrow d), (c \rightarrow b + b \rightarrow d)\} = \min\{1, 7 + \infty\} = 1$

- $d, a = \min\{(d \rightarrow a), (d \rightarrow b + b \rightarrow a)\}$
 $= \min\{6, \infty + 2\} = 6$
- $d, c = \min\{(d \rightarrow c), (d \rightarrow b + b \rightarrow c)\}$
 $= \min\{9, \infty + 5\} = 9$

Example 1: Floyd's Algorithm

Step 4: Generate $D^{(3)}$ considering the shortest path for each vertex through vertex c.



$D^{(2)} =$

	a	b	c	d
a	0	∞	3	∞
b	2	0	5	∞
c	9	7	0	1
d	6	∞	9	0

$D^{(3)} =$

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	9	7	0	1
d	6	16	9	0

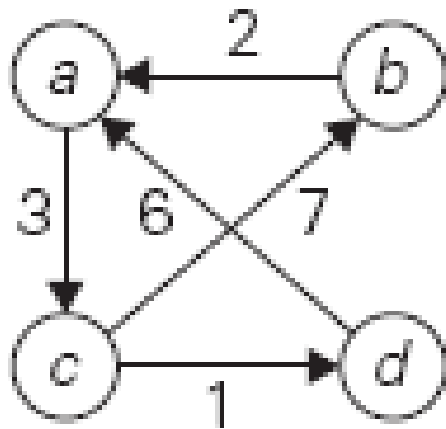
- $a, b = \min\{(a \rightarrow b), (a \rightarrow c + c \rightarrow b)\} = \min\{\infty, 3 + 7\} = 10$
- $a, d = \min\{(a \rightarrow d), (a \rightarrow c + c \rightarrow d)\} = \min\{\infty, 3 + 1\} = 4$

- $b, a = \min\{(b \rightarrow a), (b \rightarrow c + c \rightarrow a)\} = \min\{2, 5 + 9\} = 2$
- $b, d = \min\{(b \rightarrow d), (b \rightarrow c + c \rightarrow d)\} = \min\{\infty, 5 + 1\} = 6$

- $d, a = \min\{(d \rightarrow a), (d \rightarrow c + c \rightarrow a)\} = \min\{6, 9 + 9\} = 6$
- $d, b = \min\{(d \rightarrow b), (d \rightarrow c + c \rightarrow b)\} = \min\{\infty, 9 + 7\} = 16$

Example 1: Floyd's Algorithm

Step 5: Generate $D^{(4)}$ considering the shortest path for each vertex through vertex d.



$D^{(3)} =$

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	9	7	0	1
d	6	16	9	0

$D^{(4)} =$

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	7	7	0	1
d	6	16	9	0

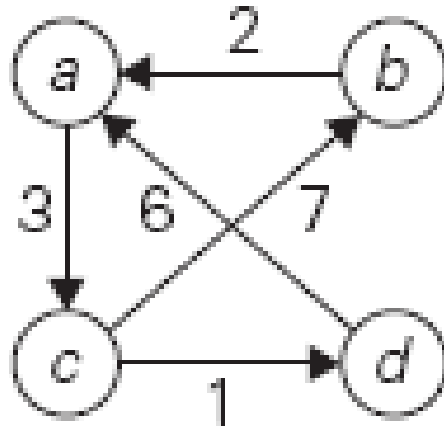
- $a, b = \min\{(a \rightarrow b), (a \rightarrow d + d \rightarrow b)\} = \min\{10, 4 + 16\} = 10$
- $a, c = \min\{(a \rightarrow c), (a \rightarrow d + d \rightarrow c)\} = \min\{3, 4 + 9\} = 3$

- $b, a = \min\{(b \rightarrow a), (b \rightarrow d + d \rightarrow a)\} = \min\{2, 6 + 6\} = 2$
- $b, c = \min\{(b \rightarrow c), (b \rightarrow d + d \rightarrow c)\} = \min\{5, 6 + 9\} = 5$

- $c, a = \min\{(c \rightarrow a), (c \rightarrow d + d \rightarrow a)\}$
 $= \min\{9, 1 + 6\} = 7$
- $c, b = \min\{(c \rightarrow b), (c \rightarrow d + d \rightarrow b)\}$
 $= \min\{7, 1 + 16\} = 7$

Example 1: Floyd's Algorithm

- Final distance matrix for the given graph:



$D^{(4)} =$

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	7	7	0	1
d	6	16	9	0

Floyd's Algorithm (pseudocode and analysis)

ALGORITHM *Floyd*($W[1..n, 1..n]$)

//Implements Floyd's algorithm for the all-pairs shortest-paths problem

//Input: The weight matrix W of a graph with no negative-length cycle

//Output: The distance matrix of the shortest paths' lengths

$D \leftarrow W$ //is not necessary if W can be overwritten

for $k \leftarrow 1$ **to** n **do**

for $i \leftarrow 1$ **to** n **do**

for $j \leftarrow 1$ **to** n **do**

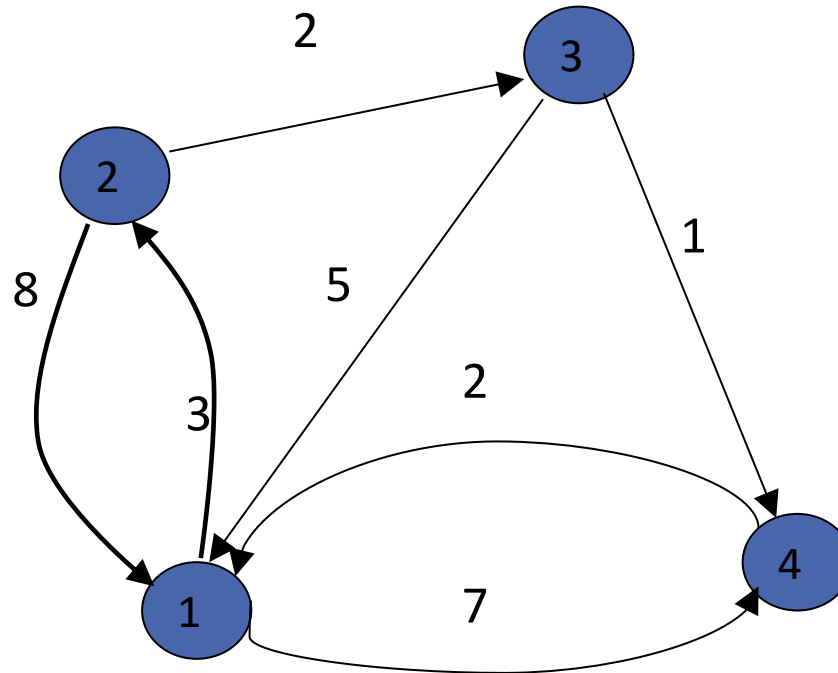
$D[i, j] \leftarrow \min\{D[i, j], D[i, k] + D[k, j]\}$

return D

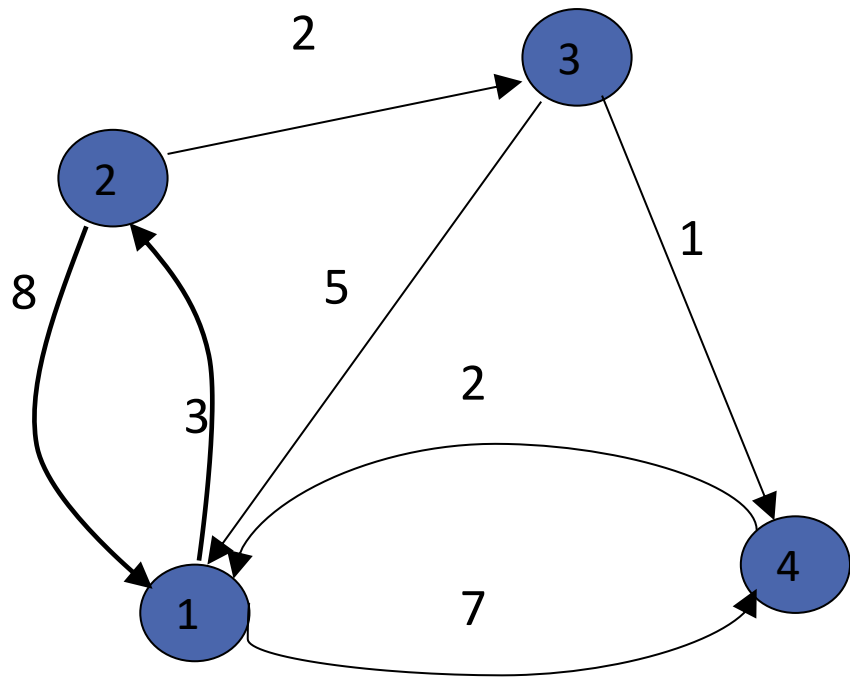
- Time efficiency: $\Theta(n^3)$
- Space efficiency: Matrices can be written over their predecessors

Example 2: Floyd's Algorithm

- Find shortest paths between every pair of vertices



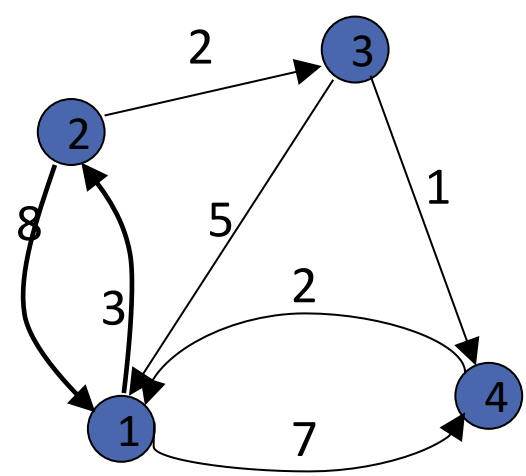
Example 2: Floyd's Algorithm



$D^{(0)} =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

Example 2: Floyd's Algorithm



D⁽⁰⁾=

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

	1	2	3	4
1	0	3	∞	7
2	8	0	2	15
3	5	8	0	1
4	2	5	∞	0

D⁽¹⁾

	1	2	3	4
1	0	3	5	7
2	8	0	2	15
3	5	8	0	1
4	2	5	7	0

D⁽²⁾

	1	2	3	4
1	0	3	5	6
2	7	0	2	3
3	5	8	0	1
4	2	5	7	0

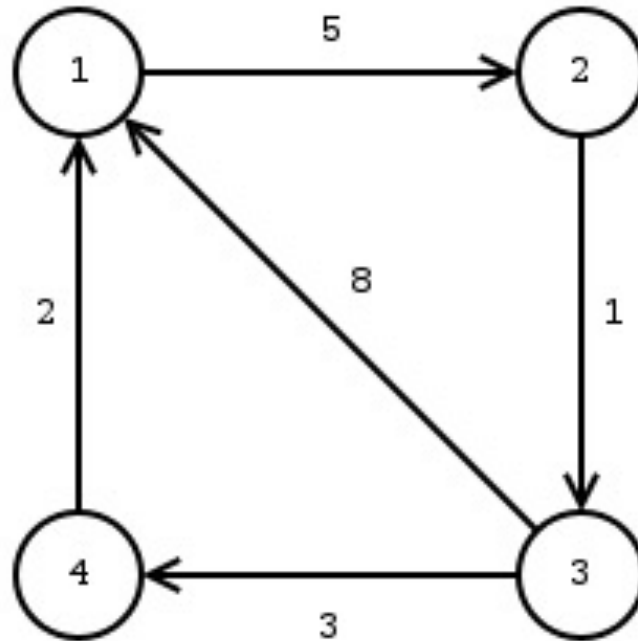
D⁽³⁾

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

D⁽⁴⁾

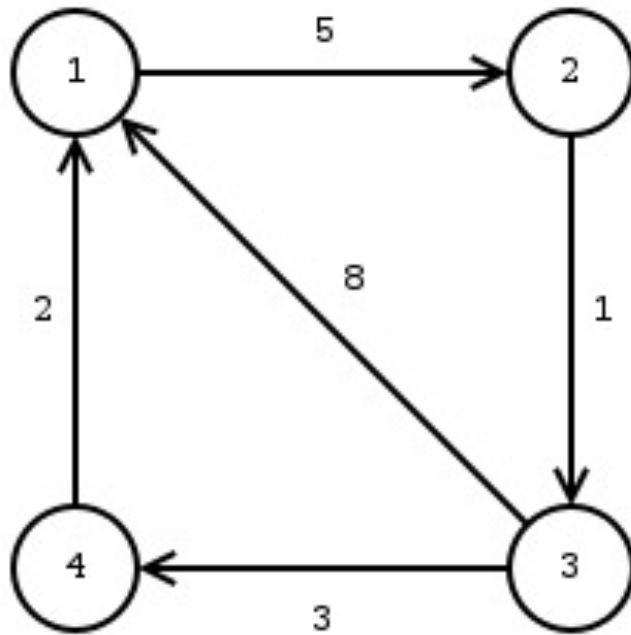
Example 3: Floyd's Algorithm

- Find shortest paths between every pair of vertices



Example 3: Floyd's Algorithm

- Solution



	1	2	3	4
1	0	5	6	9
2	6	0	1	4
3	5	10	0	3
4	2	7	8	0

$D^{(4)}$

Summary

- Dynamic programming
- Change making
- Coin collecting by robot
- Floyd's Algorithm - All pairs shortest paths

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.